Unité 7 — Injection & SQLi

Livrable B — Support de cours · UE « Software Security »

Code UE 7532 14 U32 D1 · 11 octobre 2026

> **Place dans le programme.** « Expliquer et mettre en pratique les failles les plus courantes » côté applicatif. Cette unité traite la famille des **injections** et, en détail, l'**injection SQL**. *Fondé sur meltypot (deck « Software Security » : injection, SQLi `login()`, Bobby Tables, requêtes paramétrées, pattern source/sink). Le laboratoire web (application Flask fournie) est un contenu auteur.*

1. Le motif commun à toutes les injections

Une **injection** survient quand une donnée contrôlée par l'utilisateur est insérée dans une phrase interprétée par un sous-système (SQL, shell, HTML, LDAP…) **sans séparation** entre code et données. L'attaquant glisse alors de la *syntaxe* là où le développeur n'attendait que de la *donnée*. Vocabulaire (repris en Unité 10) : une **source** (l'entrée), un **sink** (l'endroit dangereux : `execute`, `system`…), et éventuellement un **assainisseur** entre les deux.

2. Injection SQL — anatomie

Code vulnérable typique (meltypot) : la requête est fabriquée par **concaténation de chaînes**.

PYTHON/FLASK — ROUTE DE CONNEXION VULNÉRABLE

```
@app.route("/login")
def login():
    u = request.args.get("user","")
    p = request.args.get("pass","")
    q = "SELECT * FROM users WHERE user='%s' AND pass='%s'" % (u, p)  # VULN
    rows = db().execute(q).fetchall()
    return "OK" if rows else "echec"
```

Avec `user = admin'--`, la requête devient :

REQUÊTE INJECTÉE

```
SELECT * FROM users WHERE user='admin'--' AND pass='x'
```

Le `--` commente la fin : la vérification du mot de passe disparaît, on se connecte en **admin** sans le connaître. Avec `' OR '1'='1`, la condition est toujours vraie : on récupère toutes les lignes.

3. Bobby Tables & la variété des charges

L'injection ne se limite pas au contournement d'authentification : extraction de données (`UNION SELECT`), modification/suppression (`; DROP TABLE` — la célèbre blague « Bobby Tables » de meltypot), exfiltration en aveugle (blind SQLi, basée sur le temps ou les erreurs).

4. La vraie correction : séparer code et données

La seule défense robuste est la **requête paramétrée** (*prepared statement*) : la structure de la requête est figée, les valeurs sont transmises à part et ne peuvent jamais devenir de la syntaxe.

PYTHON — REQUÊTE PARAMÉTRÉE (CORRECTIF)

```
rows = db().execute(
    "SELECT * FROM users WHERE user=? AND pass=?", (u, p)   # requete parametree
).fetchall()
```

- **Requêtes paramétrées / ORM** : défense principale.
- **Moindre privilège** du compte base de données (pas de DROP depuis l'appli).
- **Validation d'entrée** (liste blanche de formats) : défense en profondeur, jamais suffisante seule.
- **Échapper** manuellement : fragile, à éviter au profit du paramétrage.

5. Détection

Revue de code et SAST (Unité 9) repèrent les concaténations vers un `execute` ; l'analyse de teinte (taint, Unité 10) relie formellement les sources aux sinks ; les tests d'intrusion confirment dynamiquement.

6. Points clés

- Toute injection = donnée utilisateur interprétée comme du code faute de séparation.
- SQLi : contournement d'auth, exfiltration, altération de données.
- Correction robuste = requêtes paramétrées (pas l'échappement maison).

Exemples SQLi issus de meltypot ; application de labo et sorties produites et vérifiées (Flask + sqlite3).